

Practical / Journal Assignment - 01 | SOLVED ANSWERS

Q1. Create a simple ASP.NET Core application and configure it to run in both In-Process and Out-of-Process hosting modes.

Theory: ASP.NET Core supports two hosting models when using IIS:

- **In-Process Hosting:** The app runs inside the IIS worker process (w3wp.exe). It is faster because there is no inter-process communication overhead.
- **Out-of-Process Hosting:** The app runs in a separate process (dotnet.exe). IIS acts as a reverse proxy forwarding requests to Kestrel.

Step 1: Create a new project

```
dotnet new web -n HostingDemo
cd HostingDemo
```

Step 2: Program.cs

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.MapGet("/", () => "Hello from ASP.NET Core!");
app.Run();
```

Step 3: Configure web.config for In-Process

```
<aspNetCore processPath="dotnet"
            arguments=".\\HostingDemo.dll"
            hostingModel="inprocess" />
```

Step 4: Configure web.config for Out-of-Process

```
<aspNetCore processPath="dotnet"
            arguments=".\\HostingDemo.dll"
            hostingModel="outofprocess" />
```

Step 5: .csproj setting (alternative)

```
<PropertyGroup>
  <AspNetCoreHostingModel>InProcess</AspNetCoreHostingModel>
  <!-- Change to OutOfProcess for out-of-process -->
</PropertyGroup>
```

Note: In-Process is the default and recommended for performance. Out-of-Process is used when isolation is needed.

Q2. Create an ASP.NET Core application which demonstrates serving static files directly from the file system.

Theory: Static files (HTML, CSS, JS, images) are served using the Static Files Middleware. Files placed in the **wwwroot** folder are automatically accessible. `UseStaticFiles()` must be called in the middleware pipeline.

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

// Enable static files middleware
app.UseStaticFiles();

app.MapGet("/", () => "Visit /index.html for static content");
```

```
app.Run();
```

wwwroot/index.html

```
<!DOCTYPE html>
<html>
<head><title>Static File Demo</title></head>
<body>
  <h1>Hello from Static File!</h1>
  <p>This file is served from wwwroot folder.</p>
  
</body>
</html>
```

After running the app, navigate to **https://localhost:xxxx/index.html** to view the static file.

Note: Without `app.UseStaticFiles()`, the browser will get a 404 error even if the file exists in wwwroot.

Q3. Create an ASP.NET Core application demonstrating Configuring Default Page using WelcomePage Middleware and UseDefaultFiles Middleware.

Theory:

- **UseWelcomePage()**: Displays a simple built-in welcome page for the root URL.
- **UseDefaultFiles()**: Remaps requests to default files like index.html, default.html. Must be called BEFORE `UseStaticFiles()`.

Program.cs — Using WelcomePage Middleware

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
```

```
// Shows built-in welcome page at root URL
app.UseWelcomePage("/welcome");
```

```
app.UseStaticFiles();
app.Run();
```

Program.cs — Using UseDefaultFiles Middleware

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
```

```
// Must come before UseStaticFiles
app.UseDefaultFiles(); // Looks for index.html by default
app.UseStaticFiles();
```

```
app.Run();
```

Place an **index.html** file inside the **wwwroot** folder. When you navigate to the root URL, it will automatically serve index.html.

Q4. Create an ASP.NET Core application which demonstrates setting a Custom HTML Page as the Default Page.

Theory: We can configure a custom file (e.g., myhome.html) as the default page using **DefaultFileOptions**.

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
```

```
// Set custom default page
var defaultFileOptions = new DefaultFileOptions();
defaultFileOptions.DefaultFileNames.Clear();
defaultFileOptions.DefaultFileNames.Add("myhome.html");
```

```
app.UseDefaultFiles(defaultFileOptions);
app.UseStaticFiles();

app.Run();
```

wwwroot/myhome.html

```
<!DOCTYPE html>
<html>
<head><title>Custom Default Page</title></head>
<body>
  <h1>Welcome to My Custom Default Page!</h1>
  <p>This is a custom default page configured in ASP.NET Core.</p>
</body>
</html>
```

Note: The file myhome.html must be placed inside the wwwroot folder.

Q5. Create an ASP.NET Core application which demonstrates Developer Exception Page Middleware.

Theory: The **Developer Exception Page** middleware shows detailed error information (stack trace, query string, headers, cookies) when an unhandled exception occurs. It should only be used in **Development** environment for security reasons.

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

// Enable Developer Exception Page only in Development
if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}

app.MapGet("/", () =>
{
    throw new Exception("This is a test exception to demonstrate Developer Exception Page!");
});

app.Run();
```

When you run the app in Development mode and navigate to the root URL, you will see a detailed exception page with stack trace information.

Note: In Production, use app.UseExceptionHandler("/Error") instead to show a user-friendly error page.

Q6. Create a simple ASP.NET Core empty application and setup MVC pattern into it.

Theory: MVC (Model-View-Controller) is a design pattern. In ASP.NET Core, MVC support is added using AddControllersWithViews() service and UseMvc() or UseRouting() middleware.

Step 1: Create Empty Project

```
dotnet new web -n MVCSetupDemo
cd MVCSetupDemo
```

Step 2: Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Add MVC services
builder.Services.AddControllersWithViews();

var app = builder.Build();
```

```
app.UseRouting();
app.UseStaticFiles();

// Map default MVC route
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Step 3: Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;

public class HomeController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

Step 4: Views/Home/Index.cshtml

```
@{
    ViewData["Title"] = "Home Page";
}

<h1>Welcome to ASP.NET Core MVC!</h1>
<p>MVC Pattern is successfully configured.</p>
```

Q7. Create ASP.NET Core MVC project with Employee Model, EmpController (Index, Create, Delete), and Views using in-memory list.

Models/Employee.cs

```
namespace EmployeeApp.Models
{
    public class Employee
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Department { get; set; }
        public string Designation { get; set; }
        public decimal Salary { get; set; }
        public string PhoneNumber { get; set; }
        public string Email { get; set; }
    }
}
```

Controllers/EmpController.cs

```
using Microsoft.AspNetCore.Mvc;
using EmployeeApp.Models;

public class EmpController : Controller
{
    private static List<Employee> employees = new List<Employee>
    {
        new Employee { Id=1, Name="Rahul Patil", Department="IT",
            Designation="Developer", Salary=55000,
            PhoneNumber="9876543210", Email="rahul@gmail.com" }
    };

    // INDEX - Show all employees
    public IActionResult Index()
    {
        return View(employees);
    }

    // CREATE - GET
    public IActionResult Create()
    {
        return View();
    }

    // CREATE - POST
    [HttpPost]
    public IActionResult Create(Employee emp)
    {
        emp.Id = employees.Count > 0 ? employees.Max(e => e.Id) + 1 : 1;
        employees.Add(emp);
        return RedirectToAction("Index");
    }

    // DELETE
    public IActionResult Delete(int id)
    {
        var emp = employees.FirstOrDefault(e => e.Id == id);
        if (emp != null) employees.Remove(emp);
        return RedirectToAction("Index");
    }
}
```

Views/Emp/Index.cshtml

```
@model List<EmployeeApp.Models.Employee>
@{ ViewData["Title"] = "Employee List"; }

<h2>Employee List</h2>
<a asp-action="Create" class="btn btn-primary">Add Employee</a>
<table class="table table-bordered mt-3">
  <thead class="table-dark">
    <tr>
      <th>Id</th><th>Name</th><th>Department</th>
      <th>Designation</th><th>Salary</th>
      <th>Phone</th><th>Email</th><th>Action</th>
    </tr>
  </thead>
  <tbody>
    @foreach(var emp in Model)
    {
      <tr>
        <td>@emp.Id</td><td>@emp.Name</td>
        <td>@emp.Department</td><td>@emp.Designation</td>
        <td>@emp.Salary</td><td>@emp.PhoneNumber</td>
        <td>@emp.Email</td>
        <td>
          <a asp-action="Delete" asp-route-id="@emp.Id"
            class="btn btn-danger btn-sm">Delete</a>
        </td>
      </tr>
    }
  </tbody>
</table>
```

Views/Emp/Create.cshtml

```
@model EmployeeApp.Models.Employee
@{ ViewData["Title"] = "Add Employee"; }

<h2>Add Employee</h2>
<form asp-action="Create" method="post">
  <div class="mb-3">
    <label>Name</label>
    <input asp-for="Name" class="form-control" />
  </div>
  <div class="mb-3">
    <label>Department</label>
    <input asp-for="Department" class="form-control" />
  </div>
  <div class="mb-3">
    <label>Designation</label>
    <input asp-for="Designation" class="form-control" />
  </div>
  <div class="mb-3">
    <label>Salary</label>
    <input asp-for="Salary" class="form-control" />
  </div>
  <div class="mb-3">
    <label>Phone Number</label>
    <input asp-for="PhoneNumber" class="form-control" />
  </div>
  <div class="mb-3">
    <label>Email</label>
```

```

        <input asp-for="Email" class="form-control" />
    </div>
    <button type="submit" class="btn btn-success">Save</button>
    <a asp-action="Index" class="btn btn-secondary">Cancel</a>
</form>

```

Q8. Create an ASP.NET Core MVC application to demonstrate ViewData.

Theory: ViewData is a dictionary object (ViewDataDictionary) used to pass data from a Controller to a View. It uses string keys and requires type casting when reading. Its scope is limited to the current request.

HomeController.cs

```

public IActionResult Index()
{
    ViewData["Title"] = "ViewData Demo";
    ViewData["Message"] = "Hello from ViewData!";
    ViewData["StudentName"] = "Priya Sharma";
    ViewData["Marks"] = 95;
    return View();
}

```

Views/Home/Index.cshtml

```

@{
    ViewData["Title"] = ViewData["Title"]?.ToString();
}
<h2>@ViewData["Title"]</h2>
<p>Message: @ViewData["Message"]</p>
<p>Student: @ViewData["StudentName"]</p>
<p>Marks: @ViewData["Marks"]</p>

```

Note: ViewData values are of type object. Explicit casting is required for complex types. e.g., (string)ViewData["Name"]

Q9. Create an ASP.NET Core MVC application to demonstrate ViewBag.

Theory: ViewBag is a dynamic wrapper around ViewData. It uses dynamic properties instead of string keys, so no casting is needed. It is easier to use but has the same limitations as ViewData (current request only).

HomeController.cs

```

public IActionResult Index()
{
    ViewBag.Title = "ViewBag Demo";
    ViewBag.Message = "Hello from ViewBag!";
    ViewBag.CourseName = "ASP.NET Core MVC";
    ViewBag.Year = 2025;
    return View();
}

```

Views/Home/Index.cshtml

```

<h2>@ViewBag.Title</h2>
<p>Message: @ViewBag.Message</p>
<p>Course: @ViewBag.CourseName</p>
<p>Year: @ViewBag.Year</p>

```

ViewData vs ViewBag Comparison:

- ViewData: ViewData["key"] = value; — requires casting
- ViewBag: ViewBag.key = value; — no casting needed (dynamic)

Q12. Create an ASP.NET Core MVC application to demonstrate TempData.

Theory: TempData is used to pass data between two consecutive requests (e.g., after a redirect). It stores data in session and automatically deletes it after reading. It is commonly used with the PRG (Post-Redirect-Get) pattern.

HomeController.cs

```
public IActionResult SetData()
{
    TempData["SuccessMessage"] = "Record saved successfully!";
    TempData["UserName"] = "Rahul";
    return RedirectToAction("ShowData");
}

public IActionResult ShowData()
{
    // TempData is available here after redirect
    return View();
}
```

Views/Home/ShowData.cshtml

```
<h2>TempData Demo</h2>
@if (TempData["SuccessMessage"] != null)
{
    <div class="alert alert-success">
        @TempData["SuccessMessage"]
    </div>
}
<p>User: @TempData["UserName"]</p>
```

Note: TempData is deleted after the next request. Use TempData.Keep("key") to retain it for additional requests.

Q13. Create an ASP.NET Core MVC application to demonstrate Strongly Typed View.

Theory: A **Strongly Typed View** is bound to a specific model class using the **@model** directive. It provides IntelliSense support, compile-time type checking, and eliminates the need for casting.

Models/Student.cs

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Course { get; set; }
    public int Marks { get; set; }
}
```

HomeController.cs

```
public IActionResult StudentDetail()
{
    Student student = new Student
    {
        Id = 1,
        Name = "Anil Kumar",
        Course = "BCA",
        Marks = 88
    };
    return View(student);
}
```

Views/Home/StudentDetail.cshtml

```
@model YourApp.Models.Student
@{
    ViewData["Title"] = "Student Detail";
}
<h2>Student Details (Strongly Typed View)</h2>
<table class="table table-bordered">
    <tr><th>Id</th><td>@Model.Id</td></tr>
```



```

<tr><th>Name</th><td>@Model.Name</td></tr>
<tr><th>Course</th><td>@Model.Course</td></tr>
<tr><th>Marks</th><td>@Model.Marks</td></tr>
</table>

```

Q14. Create an ASP.NET Core MVC application to demonstrate ViewModel.

Theory: A **ViewModel** is a class specifically created to hold data required by a View. It may combine data from multiple models. This keeps Views and Models loosely coupled and is a best practice in MVC.

ViewModels/StudentCourseViewModel.cs

```

public class StudentCourseViewModel
{
    public string StudentName { get; set; }
    public string CourseName { get; set; }
    public string CollegeName { get; set; }
    public int TotalMarks { get; set; }
    public string Grade { get; set; }
}

```

HomeController.cs

```

public IActionResult StudentInfo()
{
    var vm = new StudentCourseViewModel
    {
        StudentName = "Sneha Patil",
        CourseName = "B.Sc Computer Science",
        CollegeName = "Sangola Mahavidyalaya",
        TotalMarks = 450,
        Grade = "A+"
    };
    return View(vm);
}

```

Views/Home/StudentInfo.cshtml

```

@model YourApp.ViewModels.StudentCourseViewModel
<h2>Student Information (ViewModel Demo)</h2>
<p><b>Name:</b> @Model.StudentName</p>
<p><b>Course:</b> @Model.CourseName</p>
<p><b>College:</b> @Model.CollegeName</p>
<p><b>Total Marks:</b> @Model.TotalMarks</p>
<p><b>Grade:</b> @Model.Grade</p>

```

Q15. Create an ASP.NET Core MVC application to demonstrate use of Layout View.

Theory: A **Layout View** provides a common template (header, footer, navigation) for multiple views. It uses **@RenderBody()** to inject the child view content. Specified via **_ViewStart.cshtml** or directly in views.

Views/Shared/_Layout.cshtml

```
<!DOCTYPE html>
<html>
<head>
  <title>@ViewData["Title"] - My App</title>
  <link rel="stylesheet" href="~/lib/bootstrap/css/bootstrap.min.css" />
</head>
<body>
  <nav class="navbar navbar-dark bg-dark">
    <a class="navbar-brand" href="/">MyApp</a>
    <a class="nav-link text-white" asp-controller="Home" asp-action="Index">Home</a>
    <a class="nav-link text-white" asp-controller="Home" asp-action="About">About</a>
  </nav>

  <div class="container mt-4">
    @RenderBody()
  </div>

  <footer class="bg-dark text-white text-center p-2 mt-4">
    <p>© 2025 Sangola Mahavidyalaya</p>
  </footer>

  <script src="~/lib/bootstrap/js/bootstrap.bundle.min.js"></script>
  @RenderSection("Scripts", required: false)
</body>
</html>
```

Views/_ViewStart.cshtml

```
@{
    Layout = "_Layout";
}
```

Views/Home/Index.cshtml (Child View)

```
@{
    ViewData["Title"] = "Home";
}
<h2>Welcome to Layout Demo</h2>
<p>This content is injected into the Layout via @RenderBody().</p>
```

Q16. Create an ASP.NET Core MVC application to demonstrate use of Partial Views.

Theory: A **Partial View** is a reusable view fragment (like a component) that can be embedded inside other views. It reduces code duplication. Partial views are stored in **Views/Shared/** and named with underscore prefix (e.g., **_ProductCard.cshtml**).

Views/Shared/_ProductCard.cshtml

```
@model string

<div class="card" style="width:200px; display:inline-block; margin:10px;">
  <div class="card-body">
    <h5 class="card-title">@Model</h5>
    <a href="#" class="btn btn-primary btn-sm">Buy Now</a>
  </div>
</div>
```

Views/Home/Index.cshtml

```
<h2>Product List (Partial View Demo)</h2>

@* Method 1: Using Html.Partial *@
@Html.Partial("_ProductCard", "Laptop")

@* Method 2: Using Html.RenderPartial *@
@{ Html.RenderPartial("_ProductCard", "Mobile"); }

@* Method 3: Using partial tag helper *@
<partial name="_ProductCard" model="@("Tablet")" />
```

Q17. Create an ASP.NET Core MVC application which demonstrates Different Ways to Render Partial View.

Theory: ASP.NET Core MVC provides four ways to render a partial view:

1. **Html.Partial()** — Returns IHtmlContent synchronously
2. **Html.RenderPartial()** — Writes directly to response stream (void)
3. **Html.PartialAsync()** — Asynchronous version, returns Task<IHtmlContent>
4. **Html.RenderPartialAsync()** — Asynchronous, writes to stream
5. **<partial> Tag Helper** — Clean Razor syntax

Views/Home/Index.cshtml

```
<h2>Different Ways to Render Partial View</h2>

<h4>1. Html.Partial()</h4>
@Html.Partial("_MyPartial", "Data from Html.Partial")

<h4>2. Html.RenderPartial()</h4>
@{ Html.RenderPartial("_MyPartial", "Data from RenderPartial"); }

<h4>3. Html.PartialAsync()</h4>
@await Html.PartialAsync("_MyPartial", "Data from PartialAsync")

<h4>4. Html.RenderPartialAsync()</h4>
@{ await Html.RenderPartialAsync("_MyPartial", "Data from RenderPartialAsync"); }

<h4>5. Partial Tag Helper</h4>
<partial name="_MyPartial" model="@("Data from Tag Helper")" />
```

Views/Shared/_MyPartial.cshtml

```
@model string
<div class="alert alert-info">
    Partial View says: <b>@Model</b>
</div>
```

Q18. Create an ASP.NET Core MVC application which demonstrates use of Razor Syntax in Views.

Theory: **Razor** is a markup syntax for embedding C# code into HTML. It uses the @ symbol. Razor files have .cshtml extension. Key features include: @expression, @{code block}, @if, @foreach, @for, @while, @switch, comments, HTML helpers, etc.

Views/Home/RazorDemo.cshtml

```
@{
    ViewData["Title"] = "Razor Syntax Demo";
    string name = "Sangola Mahavidyalaya";
    int year = 2025;
    var fruits = new List<string> { "Apple", "Mango", "Banana", "Orange" };
}
```

```

}

<h2>Razor Syntax Demo</h2>

@* This is a Razor comment *@

<p>College: @name | Year: @year</p>

@* If-Else *@
@if (year >= 2020)
{
    <p class="text-success">Modern Year!</p>
}
else
{
    <p class="text-danger">Old Year</p>
}

@* Foreach Loop *@
<h4>Fruits List:</h4>
<ul>
@foreach (var fruit in fruits)
{
    <li>@fruit</li>
}
</ul>

@* For Loop *@
<h4>Numbers 1 to 5:</h4>
@for (int i = 1; i <= 5; i++)
{
    <span>@i </span>
}

@* Inline expression *@
<p>2 + 3 = @(2 + 3)</p>
<p>String Length: @name.Length</p>

```

Q19. Create an ASP.NET Core MVC application which demonstrates use of Bootstrap and jQuery in Views.

Theory: Bootstrap (CSS framework) and jQuery (JS library) are integrated into ASP.NET Core MVC via **libman** or **npm**. They are placed in the **wwwroot/lib** folder and referenced in Layout or individual views.

Installation via LibMan (libman.json)

```

{
  "version": "1.0",
  "libraries": [
    {
      "library": "bootstrap@5.3.0",
      "destination": "wwwroot/lib/bootstrap"
    },
    {
      "library": "jquery@3.7.0",
      "destination": "wwwroot/lib/jquery"
    }
  ]
}

```

Views/Shared/_Layout.cshtml (Include Bootstrap & jQuery)

```
<head>
  <link rel="stylesheet" href="~/lib/bootstrap/css/bootstrap.min.css" />
</head>
<body>
  @RenderBody()
  <script src="~/lib/jquery/jquery.min.js"></script>
  <script src="~/lib/bootstrap/js/bootstrap.bundle.min.js"></script>
</body>
```

Views/Home/BootstrapDemo.cshtml

```
@{ ViewData["Title"] = "Bootstrap & jQuery Demo"; }

<div class="container">
  <h2 class="text-primary">Bootstrap & jQuery Demo</h2>

  <div class="alert alert-success">Bootstrap Alert!</div>

  <button id="btnClick" class="btn btn-warning">Click Me</button>
  <p id="output" class="mt-2"></p>

  <table class="table table-striped table-hover mt-3">
    <thead class="table-dark">
      <tr><th>Name</th><th>Course</th><th>City</th></tr>
    </thead>
    <tbody>
      <tr><td>Rahul</td><td>BCA</td><td>Pune</td></tr>
      <tr><td>Sneha</td><td>BSc CS</td><td>Sangola</td></tr>
    </tbody>
  </table>
</div>

@section Scripts {
  <script>
    $(document).ready(function() {
      $("#btnClick").click(function() {
        $("#output").text("Button clicked using jQuery!").css("color","green");
      });
    });
  </script>
}
```

Q20. Create an ASP.NET Core MVC application to demonstrate Action Results: ViewResult, PartialViewResult, JsonResult, ContentResult, FileResult, RedirectResults, StatusResults, ObjectResult, EmptyResult.

Theory: Action Results are returned by controller actions. Each result type controls how the response is sent to the client. All implement IActionResult.

Controllers/ResultDemoController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
public class ResultDemoController : Controller
{
    // 1. ViewResult
    public IActionResult Index() => View();

    // 2. PartialViewResult
    public IActionResult GetPartial() => PartialView("_MyPartial");

    // 3. JsonResult
    public IActionResult GetJson()
        => Json(new { Name = "Rahul", Course = "BCA", Year = 2025 });

    // 4. ContentResult
    public IActionResult GetContent()
        => Content("<h2>Hello from ContentResult!</h2>", "text/html");

    // 5. FileResult
    public IActionResult GetFile()
    {
        byte[] fileBytes = System.IO.File.ReadAllBytes("wwwroot/sample.pdf");
        return File(fileBytes, "application/pdf", "sample.pdf");
    }

    // 6. RedirectResult
    public IActionResult GoToGoogle()
        => Redirect("https://www.google.com");

    // 7. RedirectToActionResult
    public IActionResult GoHome()
        => RedirectToAction("Index", "Home");

    // 8. StatusCodeResult
    public IActionResult Get404()
        => StatusCode(404);

    // 9. ObjectResult
    public IActionResult GetObject()
    {
        var data = new { Id = 1, Name = "Test" };
        return new ObjectResult(data) { StatusCode = 200 };
    }

    // 10. EmptyResult
    public IActionResult DoNothing()
        => new EmptyResult();

    // 11. NotFoundResult
    public IActionResult NotFoundDemo()
        => NotFound();
}
```

```
// 12. OkResult / OkObjectResult
public IActionResult GetOk()
    => Ok(new { Status = "Success", Message = "OK Result Demo" });
}
```

Q21. Create an ASP.NET Core MVC application to demonstrate Attribute Routing.

Theory: Attribute Routing uses **[Route]** and **[Http*]** attributes directly on controller actions to define URL patterns. It gives more control over URL structure than conventional routing.

Controllers/ProductController.cs

```
using Microsoft.AspNetCore.Mvc;

[Route("products")]
public class ProductController : Controller
{
    // GET /products
    [HttpGet("")]
    public IActionResult Index()
    {
        return Content("Product List - Attribute Routing");
    }

    // GET /products/details/5
    [HttpGet("details/{id:int}")]
    public IActionResult Details(int id)
    {
        return Content($"Product Details for Id: {id}");
    }

    // GET /products/search/laptop
    [HttpGet("search/{name}")]
    public IActionResult Search(string name)
    {
        return Content($"Searching for: {name}");
    }

    // POST /products/create
    [HttpPost("create")]
    public IActionResult Create()
    {
        return Content("Product Created via POST");
    }
}
```

Program.cs — Enable Attribute Routing

```
app.MapControllers(); // For attribute routing
// OR
app.MapControllerRoute("default", "{controller=Home}/{action=Index}/{id?}");
```

Q22. Create an ASP.NET Core MVC application to demonstrate Conventional Routing.

Theory: Conventional Routing defines URL patterns centrally in Program.cs using MapControllerRoute(). The URL pattern maps to controller and action names. It follows the convention: **{controller}/{action}/{id?}**

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
var app = builder.Build();

app.UseRouting();
```

```
// Default conventional route
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

// Custom conventional route
app.MapControllerRoute(
    name: "product",
    pattern: "shop/{controller=Product}/{action=List}/{id?}");

app.Run();
```

Controllers/HomeController.cs

```
public class HomeController : Controller
{
    public IActionResult Index()
        => Content("Home/Index - Conventional Routing");

    public IActionResult About()
        => Content("Home/About - Conventional Routing");

    public IActionResult Details(int id)
        => Content($"Home/Details/{id} - Conventional Routing");
}
```

URL Examples: /Home/Index → HomeController.Index() | /Home/Details/5 → HomeController.Details(5)

Q23. Create an ASP.NET Core MVC application to demonstrate Attribute Routing using Tokens.

Theory: Attribute routing supports **tokens** like **[controller]** and **[action]** that are replaced at runtime with the actual controller/action names. This avoids hardcoding route segments and makes routes more maintainable.

Controllers/OrderController.cs

```
using Microsoft.AspNetCore.Mvc;

// [controller] = "Order", [action] = method name
[Route("[controller]/[action]")]
public class OrderController : Controller
{
    // URL: /Order/Index
    public IActionResult Index()
        => Content("Order Index via Token Routing");

    // URL: /Order/Details/5
    [HttpGet("{id:int}")]
    public IActionResult Details(int id)
        => Content($"Order Details: {id}");

    // URL: /Order/Create
    [HttpGet]
    public IActionResult Create()
        => Content("Order Create Form");

    // Custom override - URL: /Order/all-orders
    [Route("/Order/all-orders")]
    public IActionResult AllOrders()
        => Content("All Orders List");
}
```

Note: [controller] token resolves to 'Order' and [action] token resolves to the method name like 'Index', 'Details', etc.

Q24. Create an ASP.NET Core MVC application to demonstrate Model Binding.

Theory: Model Binding automatically maps HTTP request data (form values, query strings, route data, headers, body) to action method parameters or model properties. Key binding sources include:

- [FromForm] — HTML form data | • [FromQuery] — Query string | • [FromRoute] — Route data | • [FromHeader] — HTTP headers | • [FromBody] — JSON body

Models/RegistrationModel.cs

```
public class RegistrationModel
{
    public string Name { get; set; }
    public string Email { get; set; }
    public int Age { get; set; }
    public string City { get; set; }
}
```

Controllers/BindingController.cs

```
using Microsoft.AspNetCore.Mvc;

public class BindingController : Controller
{
    // Route binding: /Binding/Details/10
    public IActionResult Details([FromRoute] int id)
        => Content($"From Route - Id: {id}");

    // Query string: /Binding/Search?name=Rahul&city=Pune
    public IActionResult Search([FromQuery] string name,
                               [FromQuery] string city)
        => Content($"From Query - Name: {name}, City: {city}");

    // Form binding
    [HttpGet]
    public IActionResult Register() => View();

    [HttpPost]
    public IActionResult Register([FromForm] RegistrationModel model)
        => Content($"Registered: {model.Name}, {model.Email}, {model.Age}, {model.City}");

    // JSON body binding (API style)
    [HttpPost]
    public IActionResult SaveJson([FromBody] RegistrationModel model)
        => Json(new { message = $"Saved: {model.Name}" });
}
```

Views/Binding/Register.cshtml

```
@model YourApp.Models.RegistrationModel
<h2>Model Binding - Registration Form</h2>
<form asp-action="Register" method="post">
    <input asp-for="Name" placeholder="Name" class="form-control mb-2" />
    <input asp-for="Email" placeholder="Email" class="form-control mb-2" />
    <input asp-for="Age" placeholder="Age" class="form-control mb-2" />
    <input asp-for="City" placeholder="City" class="form-control mb-2" />
    <button type="submit" class="btn btn-primary">Register</button>
</form>
```

Q25. Create an ASP.NET Core MVC application to demonstrate implementation of PRG (Post-Redirect-Get) Pattern.

Theory: The **Post-Redirect-Get (PRG) Pattern** prevents duplicate form submissions. When a form is submitted (POST), the server processes the data and then **redirects** to a GET request instead of returning a view directly.

This prevents the browser from resubmitting the form on page refresh.

Flow: User fills form → POST to server → Server processes → RedirectToAction (302) → GET request → Show result

Models/ContactModel.cs

```
public class ContactModel
{
    public string Name { get; set; }
    public string Email { get; set; }
    public string Message { get; set; }
}
```

Controllers/ContactController.cs

```
using Microsoft.AspNetCore.Mvc;

public class ContactController : Controller
{
    // GET: Show contact form
    [HttpGet]
    public IActionResult Index()
    {
        return View();
    }

    // POST: Handle form submission
    [HttpPost]
    public IActionResult Index(ContactModel model)
    {
        // Process the data (e.g., save to DB, send email)
        // Store success message in TempData
        TempData["SuccessMessage"] = $"Thank you {model.Name}! Your message has been sent.";
        TempData["Email"] = model.Email;

        // REDIRECT to GET action (PRG Pattern)
        return RedirectToAction("Success");
    }

    // GET: Show success page after redirect
    [HttpGet]
    public IActionResult Success()
    {
        return View();
    }
}
```

Views/Contact/Index.cshtml

```
@model YourApp.Models.ContactModel
@{ ViewData["Title"] = "Contact Us"; }

<div class="container">
    <h2>Contact Us (PRG Pattern Demo)</h2>
    <form asp-action="Index" method="post">
        <div class="mb-3">
            <label>Name</label>
            <input asp-for="Name" class="form-control" placeholder="Your Name" />
        </div>
        <div class="mb-3">
            <label>Email</label>
            <input asp-for="Email" class="form-control" placeholder="Your Email" />
        </div>
    </form>
</div>
```

```
</div>
<div class="mb-3">
  <label>Message</label>
  <textarea asp-for="Message" class="form-control" rows="4"></textarea>
</div>
<button type="submit" class="btn btn-success">Send Message</button>
</form>
</div>
```

Views/Contact/Success.cshtml

```
@{ ViewData["Title"] = "Success"; }

<div class="container mt-4">
  <div class="alert alert-success">
    <h4>Success!</h4>
    <p>@TempData["SuccessMessage"]</p>
    <p>Email: @TempData["Email"]</p>
  </div>
  <a asp-action="Index" class="btn btn-primary">Send Another Message</a>
</div>
```

Note: Without PRG pattern, refreshing the Success page would re-submit the POST request, causing duplicate data. PRG prevents this by redirecting after POST.